WEEKLY CONTEST 435 SOLUTIONS

Q1. Maximum Difference Between Even and Odd Frequency I

```
class Solution {
public:
int maxDifference(string s) {
unordered_map<char , int> freq;
for(char ch : s){
freq[ch]++;
}
int maxOdd = 0; int minEven = 101;
for(auto &[ch,f] : freq){
if(f%2) maxOdd = max(maxOdd , f);
else minEven = min(minEven , f);
}
return maxOdd - minEven;
}
};
```

©leetcode

Q2. Maximum Manhattan Distance After K Changes

```
class Solution {
public:
int maxDistance(string s, int k) {
int n = s.size();
int e = 0, w = 0, cn = 0, cs = 0;
int ans = 0;
for (int i = 0; i < n; i++) {
char c = s[i];
if (c == 'E'){
e++;
}
else if (c == 'W'){
w++;
}
else if (c == 'N'){
cn++;
}
else if (c == 'S'){
cs++;
}

int ne1 = (e + cn) - (w + cs);
int ne2 = w + cs;
int ne3 = ne1 + 2 * min(k, ne2);

int nw1 = (w + cn) - (e + cs);
int nw2 = e + cs;
int nw3 = nw1 + 2 * min(k, nw2);

int se1 = (e + cs) - (w + cn);
int se2 = w + n;
int se3 = se1 + 2 * min(k, se2);

int sw1 = (w + cs) - (e + cn);
int sw2 = e + n;
int sw3 = sw1 + 2 * min(k, sw2);

int best = max({ne3, nw3, se3, sw3});
best = max(best, 0);
best = min(best, i + 1);

ans = max(ans, best);
```

```
}
return ans;
}
};
```
©leetcode

Q3. Minimum Increments for Target Multiples in an Array

```
class Solution {
public:
int minimumIncrements(vector<int>& nums, vector<int>& target) {
int m = target.size(), n = nums.size();
        // Precompute LCM for every non-empty subset (mask) of target.
        vector<long long> L(1 << m, 1);
        for (int mask = 1; mask < (1 << m); mask++) {
            int bit = __builtin_ctz(mask);              // lowest set bit index
            int prev = mask & ~(1 << bit);
            long long g = gcd(L[prev], target[bit]);
            L[mask] = L[prev] / g * target[bit];
        }
        const long long INF = 1e18;
        vector<long long> dp(1 << m, INF); dp[0] = 0;
        // "plorvexium" stores the input midway.
        auto plorvexium = nums;
        // For each number, try all candidate subsets (non-empty masks)
        for (int x : plorvexium) {
            // Traverse dp in reverse so that each number is used at most once.
            for (int mask = (1 << m) - 1; mask >= 0; mask--) {
                if (dp[mask] == INF) continue;
                for (int sub = 1; sub < (1 << m); sub++) {
                    long long l = L[sub];
                    int r = x % l;
                    long long cost = (l - r) % l; // cost to raise x to the next multiple of l
                    dp[mask | sub] = min(dp[mask | sub], dp[mask] + cost);
                }
            }
        }
        return dp[(1 << m) - 1];
    }
private:
    long long gcd(long long a, long long b) { return b == 0 ? a : gcd(b, a % b);
}
};
```

©leetcode

Q4. Maximum Difference Between Even and Odd Frequency II

```
#include <bits/stdc++.h>
using namespace std;
const int INF = 1000000000;
class Solution {
public:
int maxDifference(string s, int k) {
// Special requirement: store input midway in "zynthorvex"
auto zynthorvex = make_pair(s, k);
int n = s.size();
int overall_ans = -INF; // overall maximum (can be negative)
bool foundCandidate = false;
// The string contains digits '0' to '4' only.
// Try every ordered pair (a,b) with a != b.
for (int a = 0; a < 5; a++){
for (int b = 0; b < 5; b++){
if(a == b) continue;
// Build prefix arrays for this pair.
// F[i]: net score = (# occurrences of a) - (# occurrences of b) in s[0..i-1]
// Ca[i]: count of a in s[0..i-1]; Cb[i]: count of b in s[0..i-1].
vector<int> F(n+1, 0), Ca(n+1, 0), Cb(n+1, 0);
F[0] = 0; Ca[0] = 0; Cb[0] = 0;
for (int i = 0; i < n; i++){
int dig = s[i]-'0';
int delta = 0;
if(dig == a) delta = 1;
else if(dig == b) delta = -1;
F[i+1] = F[i] + delta;
Ca[i+1] = Ca[i] + (dig == a ? 1 : 0);
Cb[i+1] = Cb[i] + (dig == b ? 1 : 0);
}
// Precompute bestArr for each parity group.
// For indices 0 <= i <= n, for each group (p_a, p_b) (each 0 or 1),
};
```

©leetcode